

Assignment 1 (10% of total marks)

**Due date: Saturday, 24 January 2026 by 9:00 pm (21:00 hours)
Singapore time.**

Scope

The tasks of this assignment consist of exercises in the areas of **Data Structures and Algorithms**, focusing on algorithm complexity analysis and recursion algorithms. The assignment covers the topics discussed in **Topics 1, 2, and 3**.

The assignment consists of:

- **Part A:** Theory questions to assess your understanding of the subject matter and support the programming question's objective.
- **Part B:** Programming questions requiring the implementation of concepts related to algorithms and their complexity.

Marks for each question are indicated next to the respective question. The total mark for Assignment 1 is **100 marks**.

Marks Distribution

- **Part A:** 60%
- **Part B:** 40%
- **Total Marks:** 100
- **Weightage:** 10% of the total subject mark

Assessment Criteria

Marks will be awarded for:

- **Correctness:** Solutions must meet the requirements of the question.
- **Comprehensiveness:** Solutions should demonstrate a complete understanding of the topic.
- **Appropriate Application:** Answers should reflect proper application of the materials covered in this subject.

Deliverables

Part A: Theory Questions (Questions 1 to 3)

- Type your answers using **MS Word** and save the completed document as a **PDF file**.
- Alternatively, you may write your answers by hand onto a piece of paper or using tablet. Ensure your handwriting is **neat**, then scan or save your handwritten solutions into a **PDF file**.

Part B: Programming Question (Question 4)

- Implement your solution following the standard requirements provided for the question. You may use **C, C++, Java, or Python**.
- Execute your program and take a **screenshot of the output**.
- Save your source code in **pure ASCII text format** with the appropriate file extension (e.g., **.cpp** for C++, **.java** for Java, or **.py** for Python).
- Ensure your program is appropriately documented with comments.
- Provide clear **compilation instructions** in a readme.txt file. Include batch or makefiles if applicable for C or C++ programs.
 - *Note:* If no compilation instructions are provided, your lecturer will attempt compilation based on their environment. Any failure due to incompatibility will result in a "failure to compile" outcome.

Submission Instructions

- Combine the following into a single **ZIP file** named **SolutionA1.zip**:
 - Your solutions to the theory questions (in **PDF format**).
 - Your source code for the programming question (in **ASCII text format**).
 - The screenshot of your program's output.
- Do not include subdirectories in your submission.

Part A: Theory Questions (60 Marks)

Question 1 – Algorithm Complexity Analysis (15 Marks)

a) Arrange the following functions in increasing order; that is, $f(n)$ should come before $g(n)$ in your list if and only if $f(n)$ is $O(g(n))$. Note: $n \geq 0$. Justify all your answers.

- $3^{\log_3(\log_2 n)}$,
- 2^{3n} ,
- $(n - 3)!$,
- $10n^0$,
- $\sqrt{16n^2}$,
- $\log_2(\log_2(n + 10))^n$,
- $12^{\log_2 n^2}$,
- $\sum_{k=1}^n k$,
- $2^{\log_2 \log_2 n} + \sqrt[3]{n}$,
- $\log_2((\log_2 n)^3)$

(5.0 marks)

Suggested answer:

Growth Rate	Name	Comment
$\Theta(1)$	Constant	$10n^0$ Reason: The complexity representing by the expression $10n^0 = 10 \times (n^0) = 10 \times 1 = 10$ is constant. Reason being the complexity expression $10n^0$ does not depend on n, the size of the amount of data in a process. Hence, the growth rate of complexity is independent of the amount of data in the process.

$\Theta(\log_b(\log_b n))$	Log-logarithmic (double logarithmic)	$\log_2((\log_2 n)^3) = 3 \log_2(\log_2 n)$ Reason: - Based on the logarithm property (power rule), $\log_2(a)^m = m \log_2 a$, we can rewrite the expression $\log_2(\log_2 n)^3 = 3 \log_2(\log_2 n)$. - Since the expression $3 \log_2(\log_2 n)$ involves applying the logarithm twice, it belongs to the class log of log.
$\Theta(\log_b n)$	Logarithmic	$3^{\log_3(\log_2 n)} = \log_2 n$ Reason: Recall that the logarithm property $2^{\log_2 n} = n$ (number raised to log). Hence, $2^{\log_2 \log_2 n} = \log_2 n$.
$\Theta(\sqrt[n]{n})$	Radical	$2^{\lg \lg n} + \sqrt[n]{n} = \lg n + \sqrt[n]{n}$ Reason: $2^{\log_2 \log_2 n} = \log_2 n$ $\log_2 n$ has a lower growth rate compared to radical $(\sqrt[n]{n})$. Hence, the expression has a complexity of class radical.
$\Theta(n)$	Linear	$\sqrt{16n^2} = 4n$ Reason: - Recall that $16 = 4 \times 4 = 4^2$. - Re-writing $\sqrt{16n^2} = \sqrt{4^2 n^2} = \sqrt{4^2} \times \sqrt{n^2}$ - Simplify $\sqrt{4^2 n^2}$ into $4 \times n$, because the square root of a square yields the absolute value of the variable. - Since $n \geq 0$, hence $4 \times n = 4n$.

$\Theta(n \log_2(\log_2 n))$	Linear-Double-Logarithmic	$\log_2(\log_2(n+10))^n$ Reason: - Applying the Power Rule $\log_b(a)^m = m \log_b a$, we have $\log_2(\log_2(n+10))^n = n \times \log_2(\log_2(n+10))$ - For a large value of $n+10 \approx n$, Hence, $n \log_2(\log_2(n+10)) \approx n \log_2(\log_2 n)$. - Hence, the expression $n \log_2(\log_2(n+10))$ simplifies to $n \times \log_2(\log_2 n)$. - The function n grows linearly (i.e., $\Theta(n)$), and the function $\log_2(\log_2(n))$ grows in double logarithmic order ('log of log') (i.e., $\Theta(\log_2(\log_2(n)))$). Hence, the result of the product of functions n and $\log_2(\log_2(n))$ is $\Theta(n \times \log_2(\log_2(n)))$.
$\Theta(n \log_b n)$	Linearithmic	-
$\Theta(n^2)$	Quadratic	$\sum_{k=1}^n k = 1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2} = \frac{n^2}{2} + \frac{n}{2}$ Reason: - Recall that $\sum_{k=1}^n k = 1 + 2 + 3 + \dots + n$. - The sum of the arithmetic series $1 + 2 + 3 + \dots + n$ equals $\frac{n(n+1)}{2}$. - Expand the expression $\frac{n(n+1)}{2}$ using algebra, we have $\frac{n^2}{2} + \frac{n}{2}$. - In the expression $\frac{n^2}{2} + \frac{n}{2}$, as n becomes large, the term $\frac{n^2}{2}$ dominates because it grows quadratically, while $\frac{n}{2}$ grows linearly. Hence, the asymptotic growth rate of $\sum_{k=1}^n k$ is in quadratic order, which corresponds to a quadratic class complexity.

$\Theta(n^3)$	Cubic	$27^{\log_3 n}$ Reason: - Recall that $27 = 3^3$. - Rewriting $27^{\log_3 n}$ as $(3^3)^{\log_3 n}$. - Using the exponential rule, $(a^m)^n = a^{m \times n}$, rewrite $(3^3)^{\log_3 n}$ as $(3^{\log_3 n})^3$. - Recall that the logarithm property $a^{\log_a n} = n$. Substituting $3^{\log_3 n}$ as n, Hence, $(3^{\log_3 n})^3 = n^3$.
$\Theta(a^n)$	Exponential	2^{3n} Reason: - Using the exponential rule, $(a^m)^n = a^{m \times n}$, we re-write 2^{3n} as $(2^3)^n = 8^n$. - Definition: An expression belongs to the exponential class if it has the form a^n, where a is a constant base ($a > 0$), and n is a variable in the exponent. - Hence, $8^n = (2^3)^n = 2^{3n}$ is an exponential expression, and it belongs to an exponential class.
$\Theta(n!)$	Factorial	$(n - 3)!$ Reason: - Definition: The factorial of a non-negative integer 'n', denoted by $n!$, is the product of all positive integers less than or equal to 'n'. - The expression $(n - 3)!$ Represents the factorial of the quantity $(n - 3)$. For example, if $n = 6$, then $(n - 3)! = (6 - 3)! = 3! = 3 \times 2 \times 1 = 6$. - Hence, $(n - 3)!$ is a factorial expression.

Hence, the list of the functions ordered by asymptotic growth rate is as follow:

$10n^0$,
 $\log_2((\log_2 n)^3)$,
 $3^{\log_3(\log_2 n)}$,
 $2^{\lg \lg n} + \sqrt[3]{n}$,
 $\sqrt{16n^2}$,
 $\log_2(\log_2(n + 10))^n$,
 $\sum_{k=1}^n k$,
 $27^{\log_3 n}$,
 2^{3n} ,
 $(n - 3)!$

- b) Give a Big-Oh characterization, in terms of n , of the running time of the method `myMethod()`. Count the approximate number of basic operations – assignments, comparisons, and arithmetic operations. Show all details of your answer.

```
def myMethod(int n):
    total = 0
    for i = 1 to n:      # inclusive range [1, n]
        j = 1
        while j < n:
            total = total + 1
            j = j * 2
    return total
```

(5.0 marks)

Def myMethod(int n):	
<i>total = 0</i>	1
<i>for i = 1 to n: # inclusive range [1,n]</i>	<i>n + 1</i>

$j = 1$	n
while $j < n$:	$(n \times \log_2 n) + 1$
$total = total + 1$	$n \times \log_2 n$
$j = j * 2$	$n \times \log_2 n$
return $total$	1

$$T(n) = 1 + n + 1 + n + n \log_2 n + 1 + n \log_2 n + n \log_2 n + 1$$

$$T(n) = 3n \log_2 n + 2n + 4$$

$$O(n \log_2 n)$$

c) Prove using the formal definition. Find constants c and n_0 .

- Prove: $7n^3 - 2n^2 + 8 \in \Theta(n^3)$
- Prove: $n^2 + 100n \in \Omega(n^2)$
- Prove: $\sum_{k=1}^n k \in \Theta(n^2)$
- Prove: $n \log_2 n \in \Omega(n)$
- Prove: $3n^2 + 7n + 20 \in O(n^2)$

Sample solution includes but is not limited to the following:

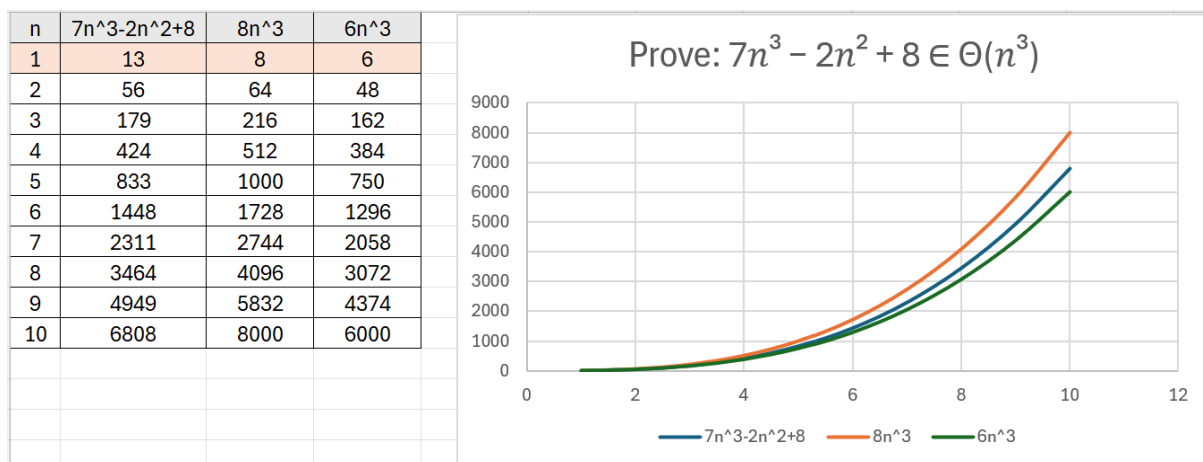
- Prove: $7n^3 - 2n^2 + 8 \in \Theta(n^3)$
 - Let $f(n) = 7n^3 - 2n^2 + 8$,
 - $g(n) = n^3$
 - Using Big-Theta definition, we prove that $c_1 g(n) \leq f(n) \leq c_2 g(n)$ for all $n \geq n_0$ and c_1 and c_2 greater than some real number.
 - Lower bound: $f(n) \geq c_1 g(n)$
 - $f(n) \geq c_1 g(n)$
 - $7n^3 - 2n^2 + 8 \geq c_1 n^3$
 - $7n^3 - c_1 n^3 - 2n^2 + 8 \geq 0$
 - $(7 - c_1)n^3 - 2n^2 + 8 \geq 0$
 - Let $c_1 = 6, n_0 = 1$, and $n = 1$
 - $(7 - 6) \times 1^3 - 2 \times 1^2 + 8 \geq 0$
 - $1 - 4 + 8 \geq 0$
 - $5 \geq 0$ - true
 - Let $c_1 = 6, n_0 = 1$, and $n = 2$
 - $(7 - 6) \times 2^3 - 2 \times 2^2 + 8 \geq 0$
 - $8 - 8 + 8 \geq 0$
 - $8 \geq 0$ - true
 - Let $c_1 = 6, n_0 = 1$, and $n = 3$

- $(7 - 6) \times 3^3 - 2 \times 3^2 + 8 \geq 0$
- $27 - 18 + 8 \geq 0$
- $17 \geq 0$ – true

- Upper bound: $f(n) \leq c_2 g(n)$
 - $f(n) \leq c_2 g(n)$
 - $7n^3 - 2n^2 + 8 \leq c_2 n^3$
 - $7n^3 - c_2 n^3 - 2n^2 + 8 \leq 0$
 - $(7 - c_2)n^3 - 2n^2 + 8 \leq 0$
 - Let $c_2 = 8, n_0 = 2$, and $n = 2$
 - $(7 - 8) \times 2^3 - 2 \times 2^2 + 8 \leq 0$
 - $-8 - 8 + 8 \leq 0$
 - $-8 \leq 0$ – true
 - Let $c_2 = 8, n_0 = 2$, and $n = 3$
 - $(7 - 8) \times 3^3 - 2 \times 3^2 + 8 \leq 0$
 - $-27 - 2 \times 9 + 8 \leq 0$
 - $-37 \leq 0$ – true

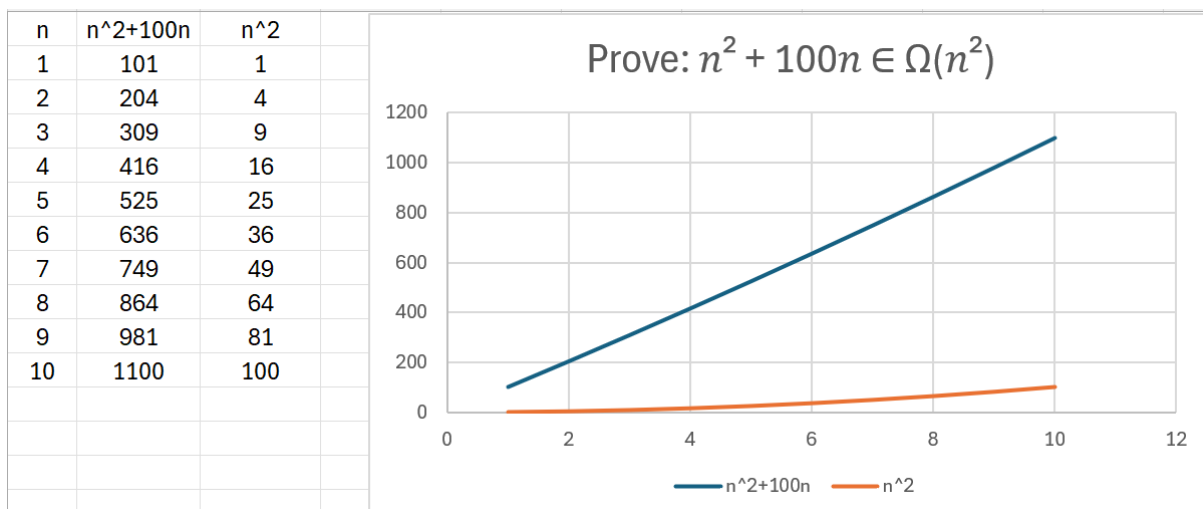
- Hence, the statement $7n^3 - 2n^2 + 8 \in \Theta(n^3)$ is valid for the lower bound condition $c_1 = 6, n_0 = 1$, and the upper bound condition $c_2 = 8$ and $n_0 = 2$.

The following graph is for verification only.



- ii. Prove: $n^2 + 100n \in \Omega(n^2)$
- Let $f(n) = n^2 + 100n$,
 - $g(n) = n^2$
 - Using Big-Theta definition, we prove that $f(n) \geq cg(n)$ for all $n \geq n_0$ and c greater than some real number.
 - Lower bound: $f(n) \geq c_1g(n)$
 - $f(n) \geq c_1g(n)$
 - $n^2 + 100n \geq cn^2$
 - $n^2 + 100n - cn^2 \geq 0$
 - $(1 - c)n^2 + 100n \geq 0$
 - Let $c = 1, n_0 = 1$, and $n = 1$
 - $(1 - 1) \times 1^2 + 100 \times 1 \geq 0$
 - $0 + 100 \geq 0$
 - $100 \geq 0$ – true
 - Let $c = 1, n_0 = 1$, and $n = 2$
 - $(1 - 1) \times 2^2 + 100 \times 2 \geq 0$
 - $0 + 200 \geq 0$
 - $200 \geq 0$ – true
 - Let $c = 1, n_0 = 1$, and $n = 3$
 - $(1 - 1) \times 3^2 + 100 \times 3 \geq 0$
 - $0 + 300 \geq 0$
 - $300 \geq 0$ – true
 - Hence, the statement $n^2 + 100n \in \Omega(n^2)$ is valid for $c = 1$ and $n_0 = 1$.

The following graph is for verification only.

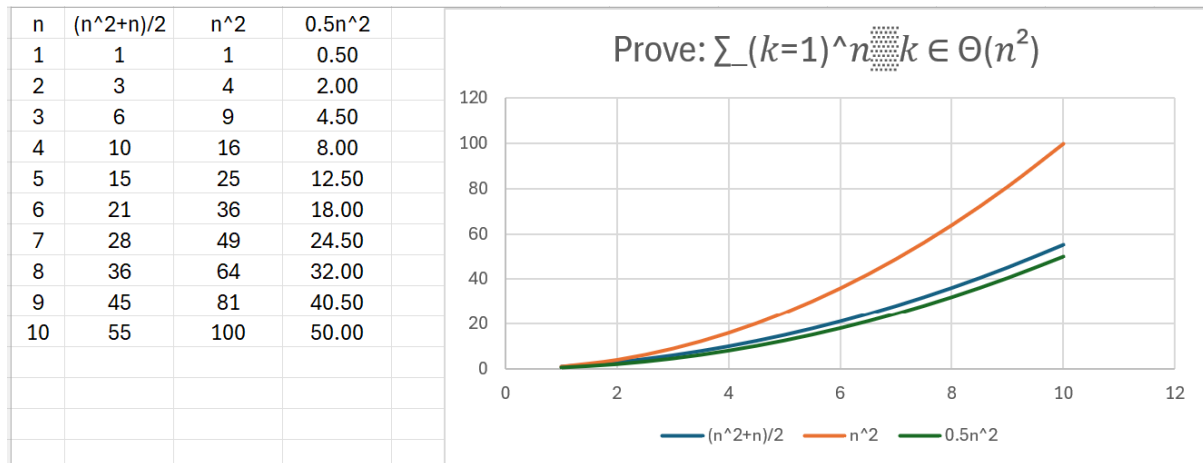


iii. Prove: $\sum_{k=1}^n k \in \Theta(n^2)$

- Let $f(n) = \sum_{k=1}^n k = \frac{n(n+1)}{2}$,
- $g(n) = n^2$
- Using Big-Theta definition, we prove that $c_1g(n) \leq f(n) \leq c_2g(n)$ for all $n \geq n_0$ and c_1 and c_2 greater than some real number.
- Lower bound: $f(n) \geq c_1g(n)$
 - $f(n) \geq c_1g(n)$
 - $\frac{n(n+1)}{2} \geq c_1n^2$
 - $n(n+1) \geq 2c_1n^2$
 - $n^2 + n \geq 2c_1n^2$
 - $n^2 + n - 2c_1n^2 \geq 0$
 - $(1 - 2c_1)n^2 + n \geq 0$
 - Let $c_1 = 0.5, n_0 = 1$, and $n = 1$
 - $(1 - 1) \times 1^2 + 1 \geq 0$
 - $0 + 1 \geq 0$
 - $1 \geq 0$ – true
 - Let $c_1 = 0.5, n_0 = 1$, and $n = 2$
 - $0 \times 2^2 + 2 \geq 0$
 - $0 + 2 \geq 0$
 - $2 \geq 0$ – true
 - Let $c_1 = 1, n_0 = 1$, and $n = 3$
 - $0 \times 3^2 + 3 \geq 0$
 - $0 + 3 \geq 0$
 - $3 \geq 0$ – true
- Upper bound: $f(n) \leq c_2g(n)$
 - $f(n) \leq c_2g(n)$
 - $\frac{n(n+1)}{2} \leq c_2n^2$
 - $n(n+1) \leq 2c_2n^2$
 - $n^2 + n \leq 2c_2n^2$
 - $n^2 + n - 2c_2n^2 \leq 0$
 - $(1 - 2c_2)n^2 + n \leq 0$
 - Let $c_2 = 1, n_0 = 1$, and $n = 1$
 - $(1 - 2) \times 1^2 + 1 \leq 0$
 - $-1 + 1 \leq 0$
 - $0 \leq 0$ – true
 - Let $c_2 = 1, n_0 = 1$, and $n = 2$
 - $(1 - 4) \times 2^2 + 2 \leq 0$
 - $-12 + 2 \leq 0$
 - $-10 \leq 0$ – true

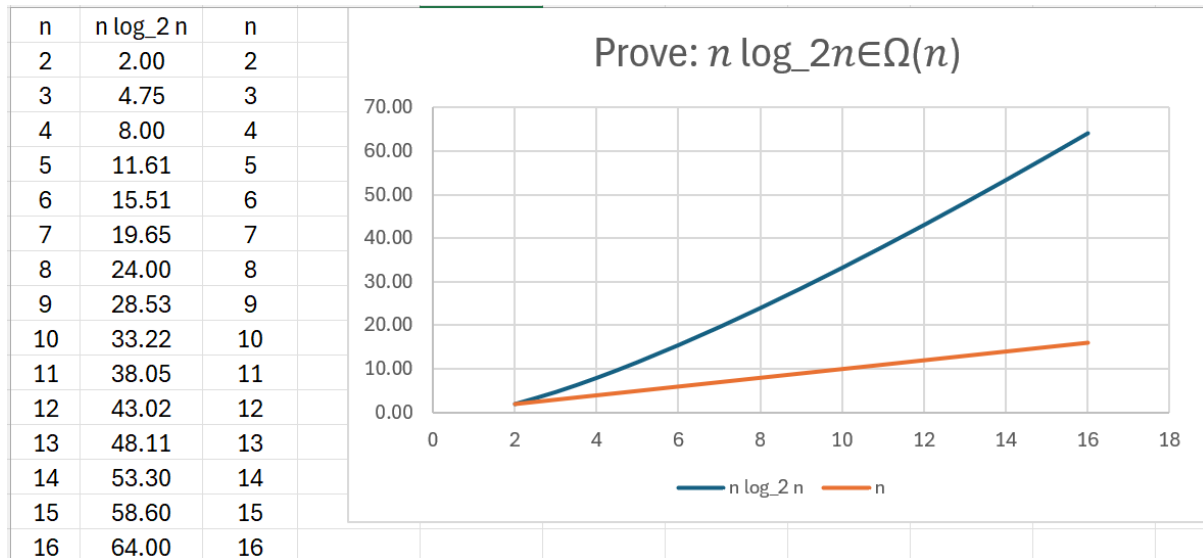
- Hence, the statement $\sum_{k=1}^n k \in \Theta(n^2)$ is valid for the lower bound condition $c_1 = 0.5, n_0 = 1$, and the upper bound condition $c_2 = 1$ and $n_0 = 1$.

The following graph is for verification only.

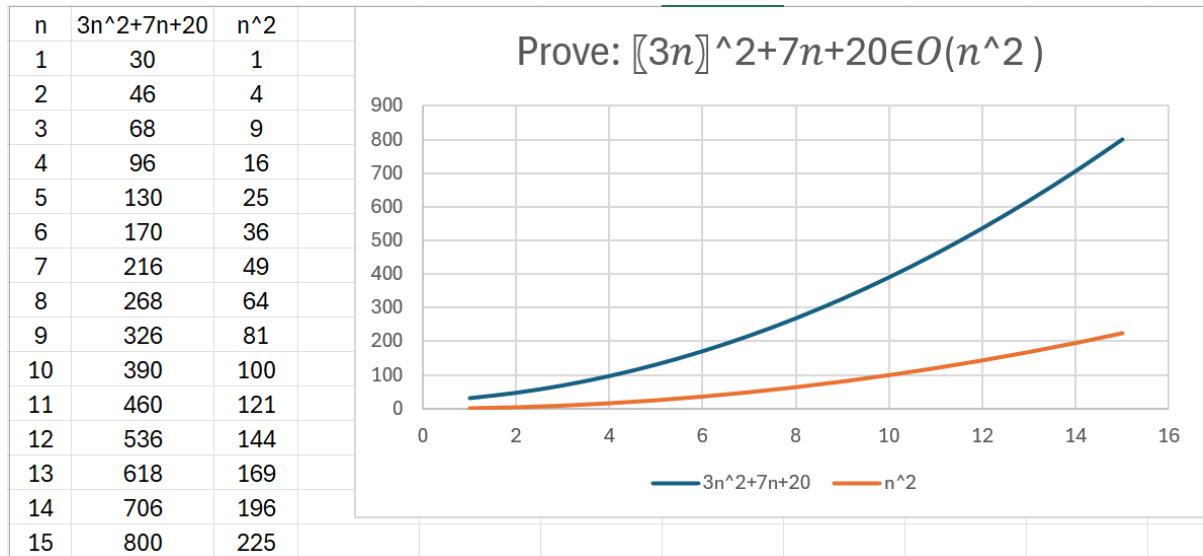


- iv. Prove: $n \log_2 n \in \Omega(n)$
- Let $f(n) = n \log_2 n$,
 - $g(n) = n$
 - Using Big-Theta definition, we prove that $f(n) \geq cg(n)$ for all $n \geq n_0$ and c greater than some real number.
 - Lower bound: $f(n) \geq c_1g(n)$
 - $f(n) \geq c_1g(n)$
 - $n \log_2 n \geq cn$
 - $n \log_2 n - cn \geq 0$
 - Let $c = 1, n_0 = 2$, and $n = 2$
 - $(2 \log_2 2) - 1 \times 2 \geq 0$
 - $2 - 2 \geq 0$
 - $0 \geq 0$ – true
 - Let $c = 1, n_0 = 2$, and $n = 4$
 - $(4 \log_2 4) - 1 \times 4 \geq 0$
 - $8 - 4 \geq 0$
 - $4 \geq 0$ – true
 - Let $c = 1, n_0 = 2$, and $n = 8$
 - $(8 \log_2 8) - 1 \times 8 \geq 0$
 - $24 - 8 \geq 0$
 - $16 \geq 0$ – true
 - Hence, the statement $n \log_2 n \in \Omega(n)$ is valid for $c = 1$ and $n_0 = 2$.

The following graph is for verification only.



- v. Prove: $3n^2 + 7n + 20 \in O(n^2)$
- Let $f(n) = 3n^2 + 7n + 20$,
 - $g(n) = n^2$
 - Using Big-Oh definition, we prove that $f(n) \leq cg(n)$ for all $n \geq n_0$ and c greater than some real number.
 - Upper bound: $f(n) \leq cg(n)$
 - $f(n) \leq cg(n)$
 - $3n^2 + 7n + 20 \leq cn^2$
 - $3n^2 + 7n + 20 - cn^2 \leq 0$
 - $(3 - c)n^2 + 7n + 20 \leq 0$
 - Let $c = 4, n_0 = 10$, and $n = 10$
 - $(3 - 4)10^2 + 7(10) + 20 \leq 0$
 - $-100 + 70 + 20 \leq 0$
 - $-10 \leq 0$
 - Let $c = 4, n_0 = 10$, and $n = 11$
 - $(3 - 4)11^2 + 7(11) + 20 \leq 0$
 - $-121 + 77 + 20 \leq 0$
 - $-24 \leq 0$



Question 2 – Recursive Algorithm (15 Marks)

Your lecturer is a funny guy; given a **sorted** list of numbers, he will use the following recursive search algorithm to search the smallest number:

```
static int minimum (L, left, right)
if (left == right) then
    return L[left]
else
    temp1 ← minimum(L, left, (left + right)/2)
    temp2 ← minimum(L, ((left + right)/2) + 1, right)
    if temp1 ≤ temp2
        return temp1
    else return temp2
```

- (a) Analyse the worst-case asymptotic complexity of the algorithm your lecturer uses. Give the worst-case running time in terms of θ notation. Explain your answer.
(10.0 marks)
- (b) How does this algorithm compare to the algorithm that simply linearly iterates through the list to look for the target? Note, you need to compare both under the worst-case running time. Explain your answer. (Hint: Tricky, do not get mis-led by the question.)
(5.0 marks)

Suggested solution includes but is not limited to the following:

- (a) The algorithm is given an array L of n sorted numbers and two parameters, the *left* and *right* pointers pointing to the first and last elements of the array. The first *if* statement will take constant time since it will return a *true* if a target is found otherwise it will return *false*; we call this time component c . This is the stopping criteria (base condition) of a recursively implemented algorithm.

The statements in the *else* block will be executed if the base condition is not met. In the block, there are two statements that will recursively call the *minimum* function. In each call, the parameters *left* and *right* are modified such that they divide the array into two halves, the left half, and the right half. The recursive calls of the *minimum* function will continue until the base condition is met. Finally, the *if* statement in the *else* block will be executed once after the recursive calls completed. Hence the worst-case asymptotic complexity is

$$T(n) = 2T\left(\frac{n}{2}\right) + 1 + c$$

Applying the master method, we see that

$$T(n) = n^{\log_b a} + c = n^{\log_2 2} + c = \theta(n).$$

Note: students can show the complexity using substitution methods, need not be using master method.

Award 6 marks for a correct analysis and correct complexity. Award partial mark up to maximum of 3 marks if the analysis and explanation are sensible with incorrect answer.

- (b) This question is to assess if the students are 'alert' about the specification of the question. Since the array is in sorted order, and the objective of the function is to find the minimum (or smallest) item, the worst-case asymptotic complexity of a simple linear search algorithm can be achieved in constant complexity, that is, the first element of the array. Hence the complexity of the algorithm specified above involve a much higher overhead than that linearly iterates through the list.

Award 4 marks for a correct analysis and explanation. Award partial mark up to maximum of 2 marks if the analysis and explanation are sensible with incorrect answer.

Question 3 – Master Theorem (30 Marks)

For each of the following recurrence relations, determine the runtime $T(n)$ complexity. Use the Master Theorem to solve the recurrence relations if the recurrence relations can be solved using Master Theorem, otherwise use expansion (or recursive substitution) to solve the recurrence relations. You must show all your working to justify your answer. Solution without clear justification or steps scores no mark.

a) $T(n) = 16T\left(\frac{n}{4}\right) + n^4$, and $T(1) = 1$ (6.0 marks)

$$a = 16, b = 4, c = 4, p = 0$$

$$\frac{a}{b^c} = \frac{16}{4^4} = \frac{16}{256} = 0.0625 < 1 \Rightarrow \text{Case 3}$$

$$\begin{aligned} p = 0 \geq 0, \therefore T(n) &= \Theta(n^c \log_b^p n) \\ &= \Theta(n^4 \log_4^0 n) \\ &= \Theta(n^4) \end{aligned}$$

b) $T(n) = 4T\left(\frac{n}{2}\right) + \frac{n^2}{\log_2 n}$, and $T(1) = 1$ (6.0 marks)

$$a = 4, b = 2, c = 2, p = -1$$

$$\frac{a}{b^c} = \frac{4}{2^2} = 1, \quad \Rightarrow \text{case 2}$$

Case 2:

(i) $p < -1$

(ii) $p = -1$, $T(n) = \Theta(n^{\log_b a} \log_b^2 n)$

(iii) $p > -1$

$$\begin{aligned} \therefore T(n) &= \Theta(n^{\log_b a} \log_b^2 n) \\ &= \Theta(n^{\log_2 4} \log_2^2 n) \\ &= \Theta(n^2 \log_2^2 n) \end{aligned}$$

c) $T(n) = 4T\left(\frac{n}{2}\right) + n^2 \sqrt{n}$, and $T(1) = 1$ (6.0 marks)

$$a = 4, b = 2, c = 5.2, p = 0$$

$$\frac{a}{b^c} = \frac{4}{2^{5.2}} < 1, \quad \Rightarrow \text{case 3}$$

Case 3:

(i) $p < 0$

(ii) $p \geq 0$, $T(n) = \Theta(n^c \log_b^p n)$

$$\begin{aligned} \therefore T(n) &= \Theta(n^c \log_2^0 n) \\ &= \Theta(n^{5.2}) = \Theta(n^2 \sqrt{n}) \end{aligned}$$

d) $T(n) = 4T\left(\frac{n}{4}\right) + n \log_2^2 n$, and $T(1) = 1$ (6.0 marks)

$$a = 4, b = 4, c = 1, p = 2$$

$$\frac{a}{b^c} = \frac{4}{4^1} = \frac{4}{4} = 1, \quad \Rightarrow \text{case 2}$$

Case 2:

(i) $p < -1$

(ii) $p = -1$

(iii) $p > -1, \quad T(n) = \Theta(n^{\log_b a} \log_b^{p+1} n)$

$$\therefore T(n) = \Theta(n^{\log_b a} \log_b^{p+1} n)$$

$$= \Theta(n^{\log_4 4} \log_4^{2+1} n)$$

$$= \Theta(n \log_4^3 n)$$

e) $T(n) = T(n-1) + n^2$, and $T(1) = 1$. (6.0 marks)

From the recurrence relation, $a = 1, b = 1$ and $f(n) = n^2$.

Since logarithmic base 1 is invalid, the Master Theorem cannot be used to test for this recurrence relation. A substitution (or expansion) technique is used instead.

$$T(n) = T(n-1) + n^2$$

$$T(n) = (T(n-2) + (n-1)^2) + n^2$$

$$T(n) = ((T(n-3) + (n-2)^2) + (n-1)^2) + n^2$$

...

$$T(n) = (T(n-k) + (n-(k-1))^2 + (n-(k-2))^2 + (n-(k-3))^2 + \dots + n^2$$

The recursive call will terminate when $n-k=0$. Hence $k=n$.

$$T(n) = T(n-n) + (n-n+1)^2 + (n-n+2)^2 + (n-n+3)^2 + \dots + (n-2)^2 + (n-1)^2 + n^2$$

$$T(n) = T(0) + 1^2 + 2^2 + 3^2 + \dots + (n-2)^2 + (n-1)^2 + n^2$$

We can express the polynomial expression of the order 2 as

$$a(n)^2 + b(n)^1 + e$$

For a worst-case scenario, the recursive function is called n times, hence, the coefficient a equals n , that is, n times of $(n)^2 = n \times (n)^2 = (n)^3$.

$T(n) = n^3 + \text{lower order terms}$.

Hence, $T(n) \in \Theta(n^3)$

Part B: Programming Question (40 Marks)

Question 4 (40 Marks)

- a) In a recent UOW-SIM orientation camp, there are m number of male students and f number of female students. Like all previous orientation camps, the number of male students, of different ages, is higher than the number of female students; that is $m > f$. The organizer of the camp would like to find out the ages among the students attending the camp that are common between both the male and female students. Device an efficient algorithm that runs on $O((m + f) \log_2 m)$ complexity, to achieve the needs of the organizer, that is, describe an algorithm (in pseudocode) that runs on $O((m + f) \log_2 m)$ to find out the ages that are common between both the male and female students. You may use the following two lists as reference when you design your algorithm.

For example:

Age_{male}:

21	19	24	22	20	23	18
----	----	----	----	----	----	----

Age_{female}:

17	17	22	19
----	----	----	----

Then, the output of your program should be 22, 19.

(15.0 marks)

Sample solution includes but is not limited to the following.

Algorithm commonAge ($mList$, $fList$)

- 1 This algorithm accepts two lists, $mList$ and $fList$.
- 2 The function will then sort the male list into ascending order using a sorting algorithm that has logarithmic time complexity, such as

quicksort. (Note, the reason I choose to sort the male list is that male list is longer, and my objective is to later search this list using binary search to achieve a $\log m$ running time complexity.)

- 3 Next, the algorithm loops through the female list, and for each element in the female list, do a binary search on the female's list. If the element (common age) is found, the algorithm will output the common age.

Pre: *mList* and *fList* are passed to the algorithm as parameter

Post: The elements that are common in the two lists.

Algorithm *commonAge(mList, fList)*

```
quicksort(mList)                                //  $O(m \log_2 m)$  complexity
For (counter = 1 to f step 1)                    // f is the size of fList
    anAge = fList[counter]                        // f
    commonAge = binarySearch(fList, anAge)      //  $f \times \log_2 m$ 
    If (commonAge != false)                      // f (worst case)
        Print commonAge
    End if
End loop
End of testForCommonElement
```

The running time complexity of the algorithm is as follow:

$$T(n) = m \log_2 m + f \log_2 m + 3f = (m + f) \log_2 m + 3f.$$

Hence, the runtime complexity of the algorithm is $(m + f) \log_2 m$.

(Note: I assume the *binarySearch(fList, anAge)* will return the common age if found and false otherwise.)

- b) Write a Java, C++, or Python program that implements the algorithm that you have come out with in Part a. You may use existing codes for quicksort algorithm or binary-search algorithm that meet the specified criteria (that is, $m \log m$ and $\log m$ run-time complexity respectively). Alternatively, you can write your own quicksort and binary-search algorithms. **(20.0 marks)**
- c) Analyze the running time complexity of your program, and include the analysis report as part of the deliverable for Part b. **(5.0 marks)**

Notes on Submission

1. This assignment is due on **Saturday, 24 January 2026 by 9:00 pm (21:00 hours) Singapore time.**
2. Submission must be made via **Moodle**. Ensure all files are combined into a single **ZIP file** without subdirectories.
3. Clearly label all components of your submission.
4. Late submissions will incur a **5% deduction per day**, including weekends.
5. Submissions more than **seven** days late will not be marked unless an extension is granted.
6. Extensions must be applied for via **SOLS** before the assignment deadline.
7. Plagiarism is treated seriously. Instances of plagiarism may result in a zero mark for the assignment.

Submit the file **SolutionA1.zip** through Moodle in the following way:

- 1) Access Moodle at **<http://moodle.uowplatform.edu.au/>**
- 2) To login use a Login link located in the right upper corner the Web page or in the middle of the bottom of the Web page
- 3) When successfully logged in, select a site **CSCI203 (SP126) Algorithms and Data Structures**
- 4) Scroll down to a section **Submissions of Assignments**
- 5) Click at **Submit** your Assignment 1 here link.
- 6) Click at a button **Add Submission**
- 7) Move the files created into an area provided in Moodle. You can drag and drop files here to add them. You can also use a link *Add...*
- 8) Click at a button **Save** changes,
- 9) Click at check box to confirm authorship of a submission,
- 10) When you are satisfied, remember to click at a button **Submit assignment.**

A policy regarding late submissions is included in the subject outline. Only one submission per student is accepted.

Assignment 1 is an individual assignment, and it is expected that all its tasks will be solved individually without any cooperation with the other students. Plagiarism is treated seriously. Students involved will likely receive zero. If you have any doubts, questions, etc. please consult your lecturer or tutor during lab classes or over e-mail.

End of specification

If you have any questions or require clarification, please contact your lecturer as early as possible. Good luck!